

OrchestratoR™ — Product Overview

Overview statement (for reading aloud):

Currently, I'm building and architecting OrchestratoR, a new and exciting AI accessibility copilot that lets product teams evaluate React component code for WCAG compliance, generate an accessibility health score, surface code-level findings and fixes, and also uses an agentic second-pass validation to verify results before release.

Currently a proof of concept scoped to buttons. This document separates what's actually built and working from what's planned, so it can be quoted accurately in interviews, pitches, and case study copy.

Last verified against the codebase: 2026-08-12.

What's built

Fully implemented and working today:

- **Accessibility health score** — evaluates a pasted React component and returns Pass / Unknown / Fail per rule, rolled up into a 0–100 score.
- **WCAG issue detection** — nine rules covering contrast, keyboard access, focus states, target size, and more, each tied to a WCAG source reference.
- **Code-level findings** — findings point to the exact line in the component code, with annotations rendered directly in the editor.
- **Suggested fixes** — each finding includes a recommended code change.
- **Live component preview** — the component renders live, in a shadow DOM, in both light and dark theme, so a fix's visual effect is immediately visible.
- **Theme-aware bugs** — the same code can score differently across themes (e.g. 85 in light, 100 in dark) when a token only fails contrast in one of them — most tools never catch this class of bug.
- **Severity / risk tagging** — each finding carries a severity level.
- **Source / evidence display** — findings show which guidance they're based on.
- **Unknown-state handling** — a rule that can't be evaluated returns "Unknown" rather than a false pass, and is excluded from the score rather than counted as either a pass or fail.
- **Score summary view** — a consolidated pass/unknown/fail summary alongside the score.

Built, with caveats

Real functionality exists, but with limits worth stating precisely rather than claiming outright:

- **A11Y DoubleCheck** and **Ask OrchestratoR** (chat) both make genuine calls to Claude for a second AI pass — but only when the local Flask backend is running with a valid API key. On the hosted demo, or without a key, both fall back to a canned response rather than a live model call. Best demonstrated locally, not from the public demo link.
- **Component-specific context** is a single static reference file injected into the prompt for the "button" component type — not a retrieval or vector-search pipeline. Accurate framing: "AI-assisted validation," not "RAG."

- **Approve / review action** — a button that records an approval in local state. It doesn't yet route to a real reviewer or persist anywhere.

Roadmap — not yet built

Planned, with no functional code yet:

- Export findings as a shareable package
- Create Jira / Linear tickets from findings
- Push findings to Figma / Figma MCP integration
- Notify designer / PM / DevOps when issues need attention
- Accessibility audit trail (history of findings, decisions, approvals)
- Design system feedback loop (surface recurring issues back to the source tokens/components)
- Developer workflow integration — CLI, CI, git hooks, or editor plugin
- Shared team visibility — multi-user roles/accounts so developers, designers, QA, and PMs see the same findings
- Support for component types beyond buttons
- Paid tier — deeper AI review, positioned against the free deterministic scoring tier

Copyright © 2026 orchestratorlabs. All rights reserved.